



Adroit
Technologies



Adroit

A Guide to Best Practices

Adroit



**MITSUBISHI
ELECTRIC**

FACTORY AUTOMATION
Authorised Distributor

COPYRIGHT

Copyright © 2025 Advanced Worx 112 (Pty) Ltd. All rights reserved.

Information in this document is subject to change without notice. The software described in this document is furnished under a license agreement or nondisclosure agreement. The software may be used or copied only in accordance with the terms of those agreements. No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or any means electronic or mechanical, including photocopying and recording for any purpose other than the purchaser's personal use without the written permission of Advanced Worx 112 (Pty) Ltd.

Advanced Worx 112 (Pty) Ltd
20 Waterford Office Park
189 Witkoppen Road (c/o Waterford Drive)
Fourways
South Africa

www.adroittech.co.za

Version	Date	Author	Comment	Approval
1.0	02.02.2018	D. Gibson	Original	
2.0	25.05.2020	J. Duncan	Update formatting	
3.0	29.06.2020	J. Duncan	Update contents, flow and Cover	
3.0	03.07.2020	D. Gibson	Review	
4.0	14.08.2020	J. Duncan	Reformat	
5.0	27.08.2020	J. Duncan	Finalise	
6.0	29.09.2021	J. Duncan	QMS	
7.0	02.02.2023	J. Duncan	Revamp	
8.0	29.05.2023	J. Duncan	Update	
9.0	20.06.2024	D. Gibson	New content and format	
10.0	04.02.2025	D. Wibberley	Content	
11.0	2025/05/20	D. Gibson	New content and format	
12.0	2025/05/29	J.Duncan	Clean up contents as per meeting 28/05/2025	
13.0	2025/06/02	D. Gibson	Add and update	
14.0	23.06.2025	J. Duncan	Finalise and QMS	D. Wibberley

Contents

1. Introduction.....	4
2. Pre-Configuration	4
2.1. Documentation	4
2.1.1. User Requirements Specification (URS).....	4
2.1.2. Functional Design Specification (FDS).....	4
2.1.3. SCADA Input Output Schedule (I/O Schedule).....	4
2.2. Pre-installation Checks	5
2.2.1. Windows Activation.....	5
2.2.2. User Account Control (UAC).....	5
2.2.3. Administrator account	5
2.2.4. Power Options.....	5
2.2.5. Regional Settings.....	6
2.2.6. Microsoft .NET Framework.....	6
2.2.7. Web Service (Optional)	6
2.2.8. Time Synchronisation Settings	6
2.3. Security.....	7
2.3.1. Security Groups and Rights.....	7
2.3.2. Software Administrator.....	7
2.3.3. Supervisor	7
2.3.4. Process Controller.....	7
2.3.5. Viewer.....	7
3. Projects Structure and Naming Conventions	8
3.1. Physical Model	9
3.1.1. Enterprise Level.....	9
3.1.2. Site Level.....	9
3.1.3. Area Level.....	9
3.1.4. Process Level	9
3.1.5. Work Unit Level.....	9
3.1.6. Equipment Level.....	9
3.1.7. Signal Level	10
3.1.8. Summary of Levels	10
3.2. Naming Convention Examples.....	10
3.2.1. Agent/Tag Naming Convention	10
3.2.2. Agent Server Naming Convention.....	12
3.2.3. PLC/Front End Device Naming Convention	12
3.2.4. Alarm Agent Naming Convention.....	12
3.2.5. Datalog Naming Convention	12
4. Adroit Configuration	13
4.1. New Configuration	13
4.1.1. Agent Server Configuration.....	14
4.1.2. Advanced Agent Server Configuration.....	14
4.1.3. Driver Configuration.....	14
5. Adroit Agent Server	14
5.1. Adding an instance of an Agent	14
5.2. Scanning Strategy	14
5.2.1. Scanning sub-system	14
5.2.2. Factors Affecting Scanning Performance	15
5.2.3. Calculating the Scan Period	15
5.3. Data Logging Strategy.....	16
5.3.1. Logging for Trends	16
5.3.2. Logging for Reports	16
5.4. Alarming Strategy	17
5.4.1. Alarm Agent Configuration.....	17
6. Adroit SmartUI Server	19
6.1. Datasources.....	19
6.1.1. Datasource configuration.....	19
6.2. Management	20
6.2.1. Security.....	20
6.2.2. Profile.....	21
6.3. Project Folder (Datasource)	21

7.	HMI Design Principles	22
7.1.	Three Primary Principles	22
7.1.1.	Clarity.....	22
7.1.2.	Consistency.....	22
7.1.3.	Feedback.....	22
7.2.	Graphic Hierarchy	22
8.	Project HMI Standards	23
8.1.	Graphic Form Layout.....	23
8.1.1.	Graphic Form Sizing.....	23
8.1.2.	Navigation Hierarchy.....	24
8.2.	Fonts.....	24
8.3.	Colours.....	24
8.3.1.	Static Colours.....	24
8.3.2.	Dynamic Colours	25

1. Introduction

This document serves as a basis against which any Adroit SCADA project can be specified and designed. Whilst it covers all the necessary standards for implementation it is intended to stimulate forethought and discussion with respect to the project requirements in order to ensure a successful project.

It is important to have an understanding of the following before proceeding:

- System Architecture
- Recommended Hardware & Software
- Network Requirements
- Licensing

For information on the above topics, please download the relevant documentation available [here](#).

2. Pre-Configuration

Before work can begin on the SCADA project, the engineer will need to understand the user requirements.

2.1. Documentation

In an Adroit SCADA project, documentation is essential in ensuring system maintainability and safety. Given the complexity and critical nature of SCADA systems accurate documentation provides a clear understanding of system architecture, configurations, communication protocols, and operational procedures. It supports efficient troubleshooting, facilitates future upgrades, and ensures compliance with industry standards and regulatory requirements. Ultimately, comprehensive documentation is key to minimizing downtime, enhancing security, and enabling seamless collaboration among engineering, operations, and maintenance teams. It is recommended that the following documents are essential to a successful completion of the solution.

2.1.1. User Requirements Specification (URS)

A URS is a description of the SCADA system to be developed and is created by the end user. It lays out functional and non-functional requirements and may include a set of use cases that describe user interactions that the software must provide.

2.1.2. Functional Design Specification (FDS)

An FDS is a formal document used to describe in detail the SCADA's intended capabilities, appearance, and interactions with users. The functional specification is a kind of guideline and continuing reference point for the SCADA engineer as he develops the operations and activities that the system must be able to perform and define how to meet the User Requirement (see User Requirements Specification).

2.1.3. SCADA Input Output Schedule (I/O Schedule)

An I/O schedule is a spreadsheet that lists all PLC and internal addresses that are required in the SCADA. The I/O Schedule, typically presented as a spreadsheet, should contain the following columns:

- PLC memory address
- PLC data type
- Agent type
- Agent name
- Agent description
- Device minimum and maximum values
- Engineering minimum and maximum values
- Alarm set points
- Is data logging required?
- Alarm Type

2.2. Pre-installation Checks

Before installing any of the Adroit products the following checks should be completed.

2.2.1. Windows Activation

Ensure that the Windows operating system on which you intend to install Adroit has been and remains legally licensed and ACTIVATED, since Adroit is not supported when it is run on an unlicensed Windows operating system.

2.2.2. User Account Control (UAC)

When installing and running Adroit on an operating system that provides User Account Control (UAC), please ensure that UAC is always ENABLED, since Adroit is not supported when it is run with UAC disabled.

2.2.3. Administrator account

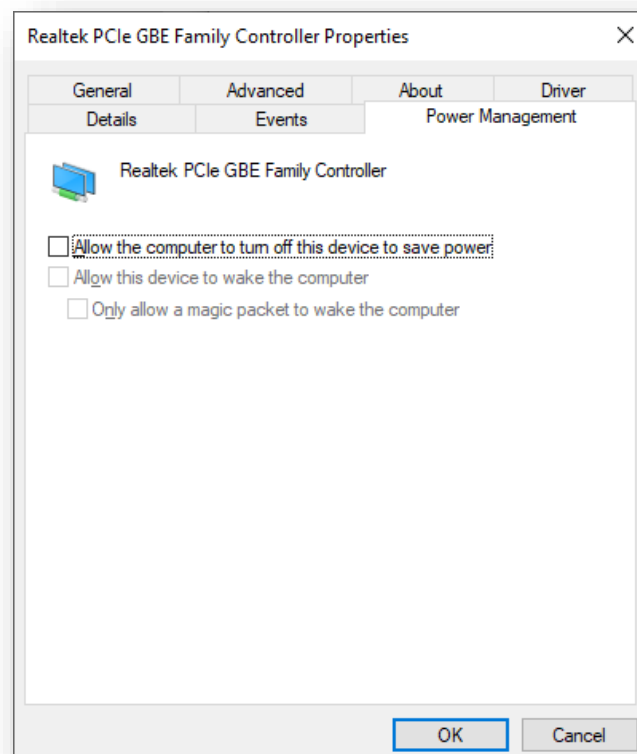
Ensure that you are not using the built-in Administrator account to run Adroit, since UAC is automatically disabled whenever this built-in Administrator account is used.

2.2.4. Power Options

All server power management options must be turned off or set to maximum were applicable. This includes:

- Hard Disks
- USB Port
- Processor
- Display

“Allow the computer to turn off this device to save power” unchecked.



2.2.5. Regional Settings

All Agent Servers and Operator stations are required to have the same regional settings. Below is an example of regional settings that need to be considered. If a property is not listed, then the default value to be used.

Language

Name	Property
Language	English (South Africa)
Headings	Grey
Static and Dynamic Labels	White
Setpoints	White

Numbers

Name	Property
Decimal symbol	. (period)
No. of digits after decimal	2
List separator	, (comma)

Time

Name	Property
Short Time	HH:mm
Long Time	HH:mm:ss

Date

Name	Property
Short date	yyyy/MM/dd
Long date	dddd, dd MMMM yyyy
First day of week	Monday

2.2.6. Microsoft .NET Framework

Adroit requires the latest Microsoft .NET framework.

2.2.7. Web Service (Optional)

If any of the Adroit Smart UI web services are to be used then the IIS (Internet Information Services) Web Service component must be installed on the server.

2.2.8. Time Synchronisation Settings

Time synchronisation is critical for all Operational Technology (OT) assets in a SCADA system because it ensures that data across all devices, such as Agent Servers, Operators (workstations), and field devices, are accurately time-stamped and aligned. Without synchronised time, discrepancies in data logs, alarms, and trend histories can occur, leading to misinterpretation of events, troubleshooting difficulties, and unreliable reports.

For example, if an operator's workstation clock is ahead of the Agent Server's, data may appear "in the future" and become greyed out, compromising real-time visibility and historical analysis. Therefore, synchronised time is essential for system integrity, event correlation, and effective decision-making.

2.3. Security

Application of security is a very important aspect of a modern SCADA/HMI. Adroit security is fully integrated into the Windows security model and a thorough working knowledge of Microsoft security is important.

The security model should be used to manage access to the system configuration, define who can do what on the system.

User security in Adroit makes use of either a Windows Domain controller or the Windows local security.

2.3.1. Security Groups and Rights

All groups will be added to the local security model on the servers and the domain user will be added to the required groups on the local servers. An associated profile will be configured for each group. The profile configuration can be found in the Profiles chapter.

NOTE : User cannot belong to multiple allowed groups used with in the Smart UI.

Below are the common levels of security groups that can be implemented.

2.3.2. Software Administrator

Highest access level, typically system architects and engineering administrators. All administrators are not groups but users. All these users should be added to the local Administrators Group on the Agent Server machines. This Group generally has complete and full access to the Adroit SCADA.

2.3.3. Supervisor

Applies to technician/maintenance level personnel.

2.3.4. Process Controller

Applies to the operators of the process. Effecting control and monitoring the system from the Operator workstation's.

2.3.5. Viewer

These users are limited to monitoring the process, but not allowed to functionally operate a process or perform actions such as Stop/Start or change the process parameters at all.

Security Matrix Example

The following are a list of functions and actions on the enterprise solution and which groups have access to said functions and actions.

Functions/Actions	Software Administrator	Supervisor	Process Controller	Viewer
Alarm Acknowledge	X	√	√	X
Set Maintenance	X	X	X	X
Comment	X	√	√	X
Designer	√	X	X	X
Documentation	√	√	√	X
Navigation level 1 (Overview)	√	√	√	√
Navigation level 2 (Area)	√	√	√	X
Navigation level 3 (Device Details)	√	√	√	X
Navigation level 4 (SP and control)	X	√	X	X
Change Setpoints	X	√	X	X
Plant Control	X	√	√	X

3. Projects Structure and Naming Conventions

This approach enables consistent naming conventions, streamlined configuration, efficient data reporting, and intuitive navigation across the SCADA system, ultimately ensuring a well-organised and maintainable solution.

A context model is a structured framework that defines how data and system elements are logically organised within the SCADA solution. It provides a hierarchical representation of the operational environment, typically based on internationally recognised standards like ISA-88 and ISA-95, to reflect the enterprise's structure from top-level business entities down to individual sensors and signals. The purpose of the context model is to simplify system design, enhance clarity, and support scalability by grouping related components meaningfully.

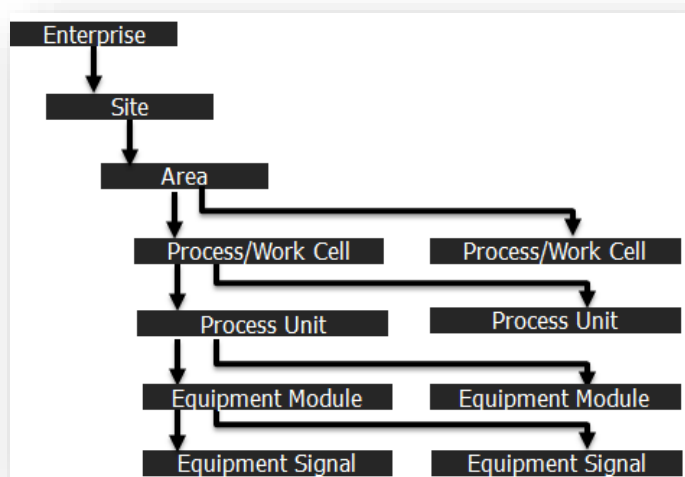


Figure 1 - S88/S95 context model

Using the one-to-many S88 structure above the engineer can define the context in the example below.

S88 Name	Adroit Context	Specified Name	Abbreviation	Related To
Enterprise	Configuration	Adroit Tech.	ADR	-
Site	Agent Server	AS 01	AS_01	ADR
Area	-	Production	PRD	AS01
Area	-	Quality Control	QCL	AS01
Process Cell	Device	PLC-A	PLCA	PRD
Process Cell	Device	PLC-B	PLCB	QCL
Unit	-	Booster Pump	BST_PMP	PRD
Equipment	-	Pump A	PMP_A	BST_PMP
Signal	-	Trip	TRP	PMP_A

Another way to define the context

Enterprise – Adroit Tech	
Site – AS 01	
Area – Production	Area – Quality Control
Process Cell – PLC-A	Process Cell – PLC-B
Unit – Booster Pump	
Equipment – Pump A	
Signal – Trip	

3.1. Physical Model

The context model is derived from a physical model that represents an enterprise's physical assets in a hierarchical structure, including enterprises, sites, areas, processes, work units, equipment, and signals. While the top two levels, enterprise and site, are defined by business needs and not explored further, they are essential for linking lower-level equipment to the broader manufacturing context. The remaining five levels focus on specific equipment types, which are groupings of physical processes and signals serving particular functions. These groupings are defined through engineering activities, where lower-level equipment is combined to form higher-level groupings to streamline operations. Once such groupings are established, they can only be changed through re-engineering.

3.1.1. Enterprise Level

An enterprise is a collection of one or more sites. It may contain sites, areas, process cells, units, equipment modules, and control modules.

3.1.2. Site Level

A site is a physical, geographical, or logical grouping determined by the enterprise. It may contain areas, process cells, units, equipment modules, and control modules.

The boundaries of a site are usually based on organizational or business criteria as opposed to technical criteria.

3.1.3. Area Level

An area is a physical, geographical, or logical grouping determined by the site. It may contain process cells, units, equipment modules, and control modules.

3.1.4. Process Level

A process contains the production units, equipment modules, and control modules required to make something.

Process control activities must respond to a combination of control requirements using a variety of methods and techniques. Requirements that cause physical control actions may include responses to process conditions or to comply with administrative requirements.

A process is a logical grouping of equipment that includes the equipment required for production of one or more products. It defines the span of logical control of one set of process equipment within an area. The existence of the process cell allows for production scheduling on a process cell basis and also allows for process cell-wide control strategies to be designed.

3.1.5. Work Unit Level

A work unit is made up of equipment modules and control modules. The modules that make up the unit may be configured as part of the unit or may be acquired temporarily to carry out specific tasks.

One or more major processing activities — such as react, crystallize, and make a solution — can be conducted in a work unit. It combines all necessary physical processing and control equipment required to perform those activities as an independent equipment grouping. It is usually centred on a major piece of processing equipment, such as a mixing tank or reactor.

Physically, it includes or can acquire the services of all logically related equipment necessary to complete the major processing task(s) required of it. Work units operate relatively independently of each other.

A work unit frequently contains or operates on a complete processing of material at some point in the processing sequence of that batch. However, in other circumstances it may contain or operate on only a portion of a batch.

This standard presumes that the work unit does not operate on more than one production process at the same time.

3.1.6. Equipment Level

Physically, the equipment module may be made up of control modules and subordinate equipment modules. An equipment module may be part of a unit or a stand-alone equipment grouping within a process cell. If engineered as a stand-alone equipment grouping, it can be an exclusive-use resource or a shared-use resource.

An equipment module can carry out a finite number of specific minor processing activities such as dosing and weighing. It combines all necessary physical processing and control equipment required to perform those activities. It is usually centred on a piece of processing equipment, such as a filter. Functionally, the scope of the equipment module is defined by the finite tasks it is designed to carry out.

3.1.7. Signal Level

A signal level is typically a collection of sensors, actuators, other control modules, and associated processing equipment that, from the point of view of control, is operated as a single entity. A signal can also be made up of other control modules. For example, a header control module could be defined as a combination of several on/off automatic block valve control modules.

3.1.8. Summary of Levels

Name	Abbreviation	Description
Enterprise	EEE	An enterprise is a collection of one or more sites. Typically not used in the naming convention
Site	SSS	A site is a physical, geographical, or logical grouping determined by the enterprise.
Area	AAA	An area is a physical, geographical, or logical grouping determined by the site.
Process Cell	PPP	A process cell contains all of the units, equipment modules, and control modules required to produce something
Unit	UUU##	A unit is made up of equipment modules and control modules.
Equipment	QQQ###	An equipment module may be part of a unit or a stand-alone equipment.
Signal	ZZZ	A control module is typically a collection of sensors, actuators, other control modules.

3.2. Naming Convention Examples

3.2.1. Agent/Tag Naming Convention

The chosen context model defines the Agents/Tag naming convention.

Each Agent Server, Device Driver, Agent type instance will need a unique name. This is to ensure that every Adroit Agent across an organisation is unique. This is of particular importance when it comes to reporting, comparing, analysing process data.

To facilitate unique naming convention the examples below has been provided.

SSS_AAA_PPP_UUU##_QQQ##_ZZZ

Note: When using Adroit 10 or older, the Agent name must not exceed 32 Character including underscores.

SSS – Site

Name	Abbreviation	Description
Durban	DURB	ABC Spice Emporium, Durban Branch
Cape Town	CAPE	ABC Spice Emporium, Cape Town Branch
Bloemfontein	BLOM	ABC Spice Emporium, Bloemfontein Branch
Johannesburg	JOZI	ABC Spice Emporium, Johannesburg Branch

AAA – Area

Name	Abbreviation	Description
Line #1	LN1	Production Line 1
Line #2	LN3	Production Line 2
Line #3	LN3	Production Line 3
Line #4	LN4	Production Line 4

Note - This section could also refer to a PLC (if multiple PLCs are used on the project)

PPP – Process

Name	Abbreviation	Description
Dispatch	DSP	Packaging and Storage
Dry Processing	DRY	Dry Mixing and Weighing
Intake	INT	Raw Material Intake and Storage
Wet Processing	WET	Wet Mixing and Weighing

UUU## – Work Unit

Name	Abbreviation	Description
Classifiers	CLA##	Where ## is an incrementing integer.
Contact tank	COT##	Where ## is an incrementing integer.
Conveyor	CON##	Where ## is an incrementing integer.
Dewatering facility	DEW##	Where ## is an incrementing integer.
Dosing facility	DOS##	Where ## is an incrementing integer.
Drying beds	DRY##	Where ## is an incrementing integer.
Lagoons	LAG##	Where ## is an incrementing integer.
Screens	SCR##	Where ## is an incrementing integer.
Thickener tanks	THI##	Where ## is an incrementing integer.

QQQ## – Equipment (All work units will be suffixed with a number to facilitate multiple work units of the same type per site)

Name	Abbreviation	Description
Flow Meter	FLM##	Where ## is an incrementing integer.
Pump	PMP##	Where ## is an incrementing integer.
Level Sensor	LVS##	Where ## is an incrementing integer.
Power Meter	PWM##	Where ## is an incrementing integer.

ZZZ – Each Signal will be matched to an equipment type.

Equipment Name	Signals	Abbr.	Description
Flow Meter	Borehole Level	LVL	
	Flow Rate	FR	
	Annual Yield to Date	AYD	
Pump	Run Time Remaining	RTR	
	Rest Time Remaining	ETR	
	Run Time Setpoint	RTS	
	Rest Time Setpoint	ETS	
	Run Hours	RH	
Level Sensor	Borehole Warning Water Level	WWL	
	Total Flow	TF	
	Borehole Stop Water Level	SWL	
	Borehole Stop Water Level Setpoint	SLS	
Power Meter	Average Voltage	AV	
	Average Current	AI	
	Average Power	AP	

As per the naming convention, an example name = **DUR_LN1_DISP_CON01_M01_STRT**

3.2.2. Agent Server Naming Convention

When running multiple Agent Servers (AS) on a LAN each AS instance requires a unique name. To facilitate this the context naming convention can be used to create a unique name for each AS.

SSS_AAA_AS##

Symbol	Abbreviation*	Example
Site	SSS	DUR
Area	AAA	LN1

As per the naming convention, the Specified Name = **DUR_LN1_AS1**.

3.2.3. PLC/Front End Device Naming Convention

Each device also requires a unique name. To facilitate this the context naming convention can be used to create a unique name for each

AAA_PPP_UUU##

Symbol	Abbreviation	Example
Area	AAA	LN1
Process	PPP	PSD
Unit	UUU##	CLA01

As per the naming convention, the example name = LN1_PSD_CLA01.

3.2.4. Alarm Agent Naming Convention

If multiple Alarm Agents are required, then the Alarm Agent instance can use the following naming convention:

SSS_AAA_ALAM##

Symbol	Abbreviation*	Example
Site	SSS	DUR
Area	AAA	LN1

As per the naming convention, the example name = DUR_LN1_ALAM01.

3.2.5. Datalog Naming Convention

When creating the datalog files, typically referred to as LGD files or log files, it is recommended to split the log files by equipment. To facilitate this the context naming convention can be used.

AAA_PPP_UUU##_QQQ##

Symbol	Description*	Abbreviation
Area	AAA	LN1
Process	PPP	DSP
Unit	UUU##	CLA01
Equipment	QQQ##	FLM01

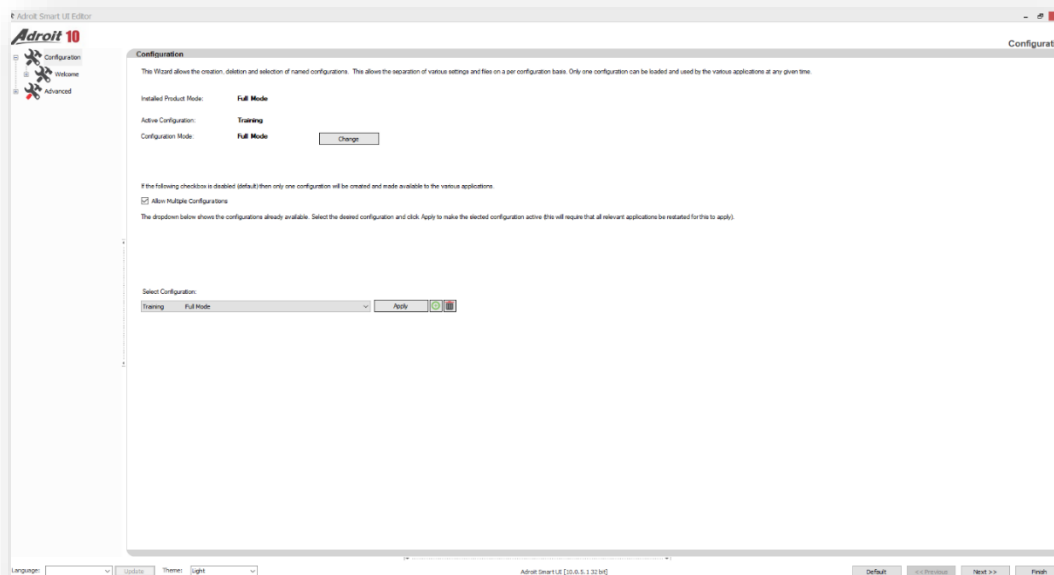
As per the naming convention, the example name = LN1_DSP_CLA01_FLM01

4. Adroit Configuration

Now that all the preliminary planning has been completed the engineer can begin to set up the Adroit SCADA software. The setup will be done using the Adroit Config Editor Application.

4.1. New Configuration

A new configuration will be created using the specified naming convention. This will allow for a new project file structure for ease of backup and restore. All project files and documentation will be stored in this file structure. It is recommended NOT to use the Default but create a new configuration for the actual project using the defined [Project Structure and Naming Convention](#).



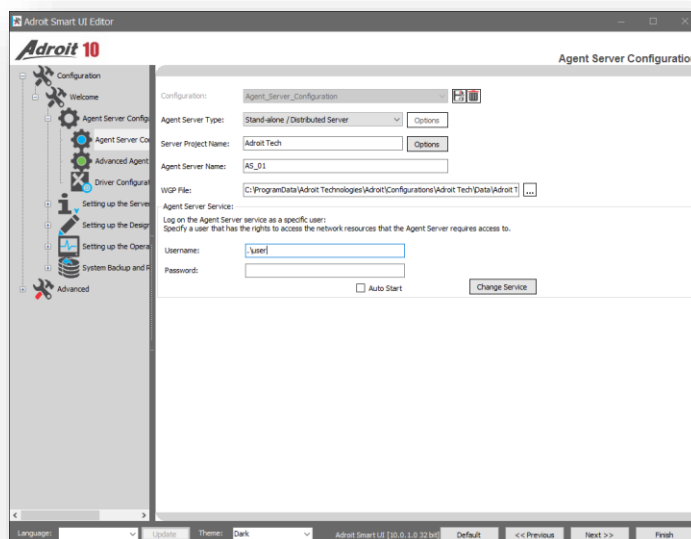
The project files will be created in the following paths:

Agent Server project files are stored in the file path:

C:\ProgramData\AdroitTechnologies\Adroit\Configurations\Project_Name

SmartUI project files are stored in the file path:

C:\ProgramData\AdroitTechnologies\Adroit\SmartUI\Configurations\Project_Name



4.1.1. Agent Server Configuration

Naming of the items below will be derived using the defined [Naming Convention](#).

- Agent Server type
- Project name
- Cluster name

4.1.2. Advanced Agent Server Configuration

The Global OLE DB connection string will be set to a SQL Server instance database. This connection string can be used by all Agents that require a SQL connection like the DBLog Agents, OEE Agent, Shift Agent and the Alarm Management Agent.

4.1.3. Driver Configuration

A mix of different protocols can be used to communicate to the PLC/FED hardware. When creating an instance of the protocol, called a device, the engineer should use the defined [Naming Convention](#).

5. Adroit Agent Server

The Agent Server (AS) contains the 3 main functions of a SCADA IO Server:

- Scanning
- Data Logging
- Alarming

5.1. Adding an instance of an Agent

Each Agent type instance will need a unique name. Naming of the Agents will be derived using the defined [Naming Convention](#).

All names and abbreviation will also be documented in the IO Schedule document.

5.2. Scanning Strategy

This is a critical aspect of setting up any Adroit system. It is important to plan the PLC addressing and Areas to be used up front so that the communications driver fetches "blocks" of data rather than communicating to many different and separated address within a PLC.

NOTE: This is more critical for large, complex sites. However, it is an important aspect of any project configuration.

Other considerations:

- Put all reporting data into addresses of the same type and are contiguous.
- Put all data for fast scanning, slow scanning in contiguous areas

5.2.1. Scanning sub-system

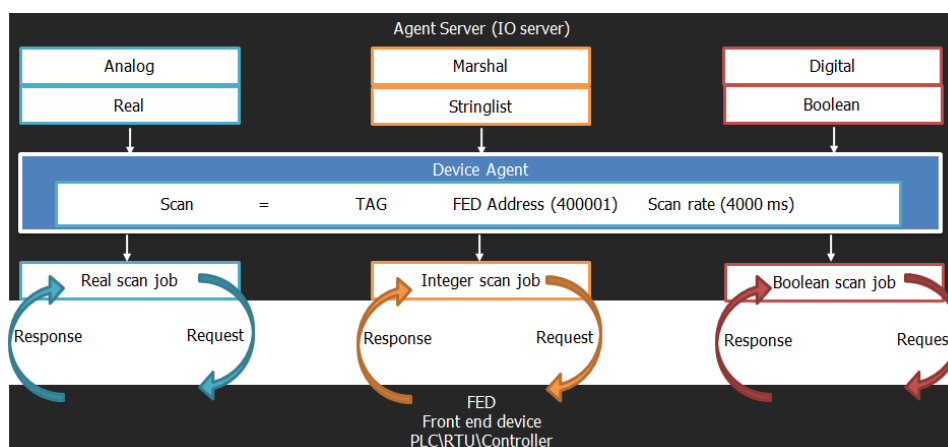


Figure 2 - Scanning subsystem - Solicited/Polled Driver Architecture

5.2.2. Factors Affecting Scanning Performance

- Bandwidth and other medium-specific issues.
- The number of devices per communications channel.
- Total number of transactions required to refresh each device on the network – contiguous data packing.
- The frequency of transactions (Scan Rate).

5.2.3. Calculating the Scan Period

It is very easy to oversubscribe (scan) to data, there are limitations to any communications medium, the document will assume serial-based communications, as it's the one most easily overestimated.

A few assumptions will have to be made:

- The total number of Outstations/PLC's/FED's on the network.
- The total number of I/O scan tasks per outstation (approx.) and their size in characters (approx.)
- Approximate/Average turnaround time for a transaction. (see diagram)
- Network characteristics such as baud rates/bandwidth etc.
- Assume for the ideal case, i.e. No errors, retries, pre-transmission delays, latency delays etc.

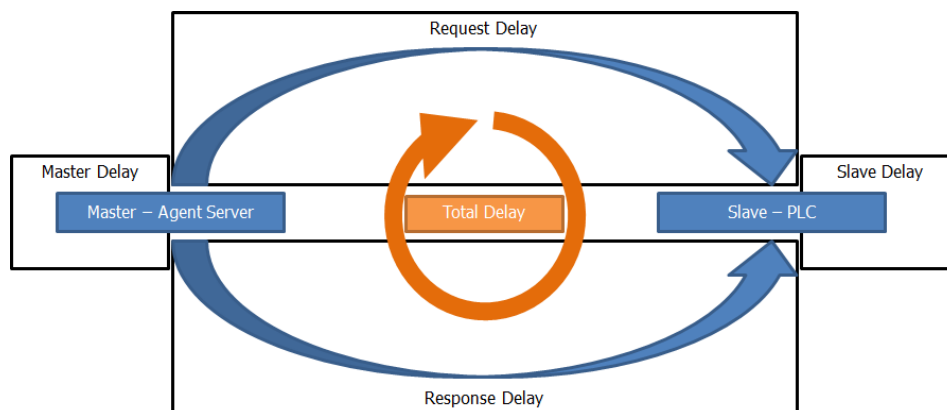


Figure 3 - Scanning transaction time

Example - Serial Communication Calculation

9600 baud rate, 60 outstations, 2 (100 chars) Transactions per outstation per refresh, and a total turnaround of 300 milliseconds per transaction. The 100 chars referred to above includes the request and response and any acknowledgements.

9600 allows 1 bit every 0.0001 seconds, assuming 1 start bit and 2 stop bits, 11 bits per character, means ~872 characters per second on the channel if the channel runs at maximum. $60 \text{ (outstations)} \times 2 \text{ (transactions)} \times 100 \text{ (chars)} = 12000 \text{ chars to refresh the entire network. } 12000 \text{ (chars in total)} / 872 \text{ (channel config max)} = 13.72 \text{ Seconds to refresh the entire network at maximum channel capacity.}$

100 characters per transaction roughly use 114 milliseconds network time but there will be a turnaround time conservatively assumed above as 300 milliseconds thus: $300 * 120 \text{ (total network transactions)} = 36 \text{ seconds to refresh the entire network.}$

Now this means if every transaction is an equal priority and assuming no errors, delays, or pre-transmission delays etc. no scan rate should exceed 36 seconds.

Implement the following solutions, to shorten the scan rate:

- Add additional 3 channels, the network update time will now be 9 Seconds.
- Prioritise scan tasks, use a slower scan rate for data that updates slower in the field, e.g. reservoir levels (e.g. Integrity polls), high priority data faster (e.g. Digital status indications).
- Use an exception based protocol where the fastest way to get a change of state data is via Unsolicited transactions. Then the other data is divided up into priorities just like option 2 above.

Every effort is to be made when drawing up the I/O schedule of the PLC/SCADA and to define logical and contiguous data areas to be scanned from Devices. This makes for efficient, reliable and fast scanning. Take note of the driver documentation that will assist in creating an optimum scanning (and hence performance) of the Adroit scanning system.

5.3. Data Logging Strategy

Data logging is a vital component of any Adroit system, providing time-based historical data that can be used to analyse, understand, and optimize production processes, and in some industries, such as food manufacturing, it's a statutory requirement for traceability. Before implementation, it's crucial to define logging requirements for each Agent/Tag. Balancing the need for fast, short-term data history (useful for process engineers) with the practicality of long-term production and statutory data storage needs.

Adroit supports flexible logging strategies, with short-term data stored in native datalog files for quick access and long-term data directed to an OLEDB-compliant database for durability and compliance.

5.3.1. Logging for Trends

The Adroit data logging sub-system is used to store/log historical data for trending purpose. The data log should be configured as follows.

- **Datasource** – A data log Agent logs the data to a .lgd file, all required signals belonging to an equipment model should be logged to the same .lgd file. The .lgd file name should be made up using the context model.
- **Length** – The length of time required to store the data. The length of time should be kept to a minimum as defined by how long the operation of the system requires. Typically this should be no longer than 3 months, anything longer than this period would be defined as long term data and should be logged to SQL.
- **Deadband Time** – The time or sample rate of a datalog should be faster than the scan rate. if your scan rate is 1000 ms then the datalog time should be 900 ms.
Using the length and deadband time the Adroit Agent Server will calculate the required size of the .lgd file. The .lgd file can be no larger than 2 GB so care should be shown when specifying the length and deadband time.

5.3.2. Logging for Reports

The DBLog Agent should be used to log data to SQL database for long term storage of data. There should be a DBLog Agent for each data type to be logged:

- **DIG_DBLog** – All Digital Agents and marshal Agent bits should be added to this DBLog Agent. The logging method should be set to Log all on change Alternatively use the log on change to minimise database entries.
- **ANA_DBLog** – All Analog Agents should be added to this DBLog Agent. The logging method should be set to Log all on interval. The logging interval is usually user defined.
- **STR_DBLog** – All String Agents and slot that's data type is a string should be added to this DBLog Agent. This Agent could be set to trigger on an event. An example of this is using a shift Agent to trigger on shift change.

Each DBLog Agent should have its own database (db) table. It is recommended that the table name should be the same name as the DBLog Agent instance.

5.4. Alarming Strategy

An effective SCADA alarming strategy is essential for ensuring safe, efficient, and reliable operation of industrial systems. Alarms serve as critical indicators that notify operators of abnormal conditions or potential issues within the process. A well-designed strategy helps prioritize alarms based on severity, operational impact, enabling timely and appropriate responses. It also reduces alarm fatigue by minimizing nuisance or irrelevant alarms, ensuring that only meaningful and actionable alerts reach the operator. The strategy should include clear guidelines for alarm configuration, categorization, and response procedures, as well as ongoing evaluation and optimization. By implementing a structured and thoughtful alarming strategy, organizations can improve situational awareness, reduce downtime, and enhance overall process safety and performance.

A 5-tier priority alarming strategy can be used as a starting point to separate the solutions events and alarms into the relevant categories with lowest priority (P0) being events and highest priority (P5) being system critical alarms.

The association of event and alarm priority will be assigned in the IO schedule by the Plant Engineer. The alarm colours for each priority can be found in the [Colours](#) chapter.

5.4.1. Alarm Agent Configuration

When configuring the Alarm Agent, it is important to have a clear understanding of the purpose for each alarm route. So, assigning priorities and outputs to each route should be documented for future proofing the solution as per the example below.

Route	Priority	Route Output/Configuration/Actions
0	-	are reserved for system alarms such as software faults and communication errors
1	-	are reserved for system alarms such as software faults and communication errors
2	Lowest (P0)	Alarm Management, not to be acknowledged, no action required
3	Low (P1)	Alarm Management, acknowledged by Operator, no action required
4	Normal (P2)	Alarm Management, acknowledged by Operator. Action to be taken within 3 days
5	High (P3)	Alarm Management, acknowledged by Operator. Action to be taken within 24 hours
6	Highest (P4)	Alarm Management, acknowledged by Operator. Action to be taken immediately, send notification to Supervisor

If multiple Alarm Agents are required, then the Alarm Agent instance can be named using the defined [Naming Convention](#).

Alarm Types

Creating custom alarm types allows for a greater degree of alarm customization and gives the operator more details around the state of the solution.

The following are examples of how to specify custom alarm types:

Agent Type	Type Name	Header slot	Priority
Analogue	Reservoir 1 Lvl High	High	P4
	Reservoir 1 Lvl Low	Low	P2
	Tower 1 Lvl High	High	P4
	Tower 1 Lvl Low	Low	P2
	RSSI Signal High	High	P3
	RSSI Signal low	Low	P3
	Voltage high	High	P3
	Voltage Low	Low	P3

Agent Type	Type Name	Header slot	Priority
Marshal	TX Fail (B#)	One for each slot were # is bit number = False	P3
	GPRS TX Fail (B#)	One for each slot were # is bit number = False	P3
	Mains Fail	Bit14 = False	P4
	Intruder	Bit15 = False	P3
	Pump 1 Trip	Bit01 = True	P4
	Pump 2 Trip	Bit04 = True	P4
	Pump 3 Trip	Bit07 = True	P4
	Pump 4 Trip	Bit10 = True	P4

Alarm List

Alarm lists are a great way to display specific alarms by routes. Alarm list can be created for equipment, area and alarm priorities. The following are examples of alarm lists and how they are allocated

Name	Agent Name	Description	Assigned Route
Global Alarm List	GLBL_ALAL	A global alarm list that will contain a list of all alarms for the system	All Routes
Site Alarm List	SSS_ALAL	A regional alarm list that will display all alarm assigned the specified region.	Routes designated to a Site
Area Alarm List	SSS_AAA_ALAL	Site specific alarm list that will display the alarms associated with a specific site.	Routes designated to an Area

Alarm Management Agent

The Alarm Management Agent is used to route alarms to a database. This can be used simply to store historical alarms that can be viewed in the Historical Alarm control in the Operator or when fully licenced can be used with the reporting and analytics available in the Alarm Management and Analysis software suite (AMA)

When setting up the Alarm Management Agent the following configuration choices are recommended:

1. Use the global OLEDB connection, to connect to the SQL instance.

Note: If a fully licensed version of Alarm Management is being used, then the Context model can be used to build the categories and sub-categories. Make sure to add alarm instances to the corresponding categories or sub-categories. Only enable Reasons and Notes for Alarm instances that are defined as high priority alarm types.

6. Adroit SmartUI Server

The Adroit SmartUI Server is the middleware layer that manages connectivity to data and the graphic forms used in the Operator environment.

This chapter focuses on applying structured design principles (clarity, consistency and feedback) to ensure that the User Interface (UI) accurately reflect real-time system states, support efficient user workflows and reduce cognitive Operator load during operation.

By leveraging context-aware graphics, modular components and standardised navigation hierarchies, engineers can create scalable interfaces that promote situational awareness and minimise user error. The guidelines presented here are informed by best practices in control system design and are intended to streamline both development and runtime performance across diverse industrial applications.

6.1. Datasources

The SmartUI Server has a number of Datasource types, which allows for the collection of Datasource like the Agent Servers and historical data storage (SQL Server).

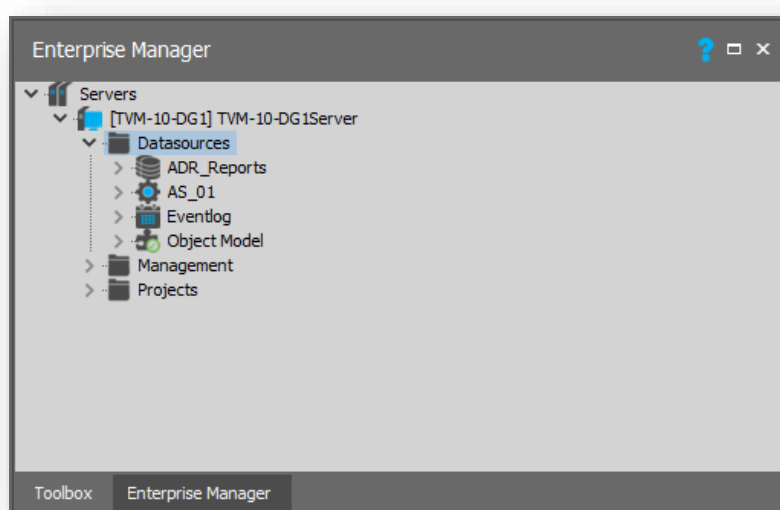


Figure 4 - SmartUI Datasources

6.1.1. Datasource configuration

When creating a Datasource, the name of the Datasource should reflect what it is connecting to. For example, the name for the Adroit Datasource should be the same as the Agent Server's name and the OLE DB Datasource name should be the database's name that it connects to.

OLE DB

An OLE DB Datasource exposes the data contained within an OLE DB compliant database or data store, such as MS SQL Server. This data is contained within fields in one or more tables. When using the OLE DB datasource to query the db's data (especially when creating custom queries) the following should be taken into account:

- When naming your query a good practise is to prefix the name with the query type, the second part should be a short description about the queries purpose.
Example: **Select_ProductByID.**
- Confirm that the query runs in SQL Server management studio.
- When parametrizing the query, make sure to use a name that reflect what is being constrained.
Example:



Figure 5 - Parametrized select Query

6.2. Management

Before configuring security and the profiles under management is highly recommended that the Windows security's Users and Groups strategy has been implemented. See the [Security](#) chapter.

6.2.1. Security

Before setting up any security configuration of the Datasources it is recommended to add all required groups to the Allowed Groups

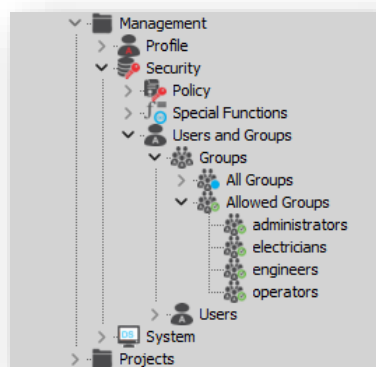


Figure 6 - Security configuration Allowed Groups

A phased approach should be taken when implementing the predefined security strategy. Assigning the Group with the highest security access to the Datasources, then proceed to the Group with the next level in a descending order.

6.2.2. Profile

When creating profiles, a simple design is better than a more complex one. First determine how many user experiences you require. As an example most users could load a single overview Graphic Form that allows navigation to the rest of the solution. So, a single profile that applies to all users can be used. However, if a group of users requires access to Graphic Forms that are designed to run on a mobile device then a second profile called "*Mobile Users*" can be created. If multiple languages are used in the solution, then a profile per language should be created.

6.3. Project Folder (Datasource)

The Projects folder is the repository for all Graphic Forms used to visualise data in the Operator and should reflect the Context Model of the site. This will assist in maintenance of the project and allow easy access to forms used. Makes sure to remove any Graphic Form project(s) that are not used in the final solution.

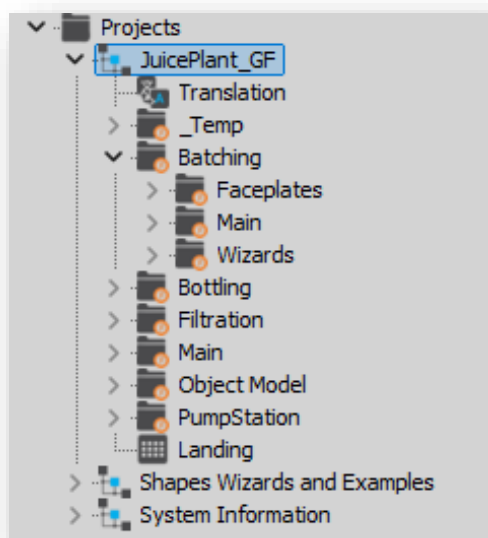


Figure 7 - Graphic form project Structure

7. HMI Design Principles

Much has been learned over the years regarding good and bad HMI design principles/practices. Below are three primary principles which need to be implemented for an efficient HMI design.

7.1. Three Primary Principles

7.1.1. Clarity

- Graphics are easy to read and intuitively understandable.
- Graphics show the process state and conditions clearly.
- Graphic elements used to manipulate the process are clearly distinguishable and consistently implemented.
- Graphics do not contain unnecessary details and clutter, including too many different colours
- Graphics convey relevant information, not just data.
- Information has prominence based on relative importance.
- Alarms and indications of abnormal situations are clear; prominent; and consistently distinguishable.

7.1.2. Consistency

- Graphics functions are standardised, intuitive, straightforward and involve minimum keystrokes or pointer manipulations.
- The HMI is set up for navigation in a logical, hierarchical and performance-oriented manner.

7.1.3. Feedback

- Graphic elements and objects (wizards/controls) must behave and function consistently in all graphics and all situations.
- Important actions with significant consequences will have confirmation mechanisms to avoid inadvertent activation.
- Design principles will be used to minimize user fatigue, since operations personnel use the graphics constantly.

7.2. Graphic Hierarchy

High-performance graphics are designed in a hierarchy for progressive disclosure of further details.

- Level 1 – Process Area **Overview**
- Level 2 – Process Unit **Control**
- Level 3 – Process Unit **Detail**
- Level 4 – Process Unit **Support and Diagnostic Displays**

8. Project HMI Standards

How to apply the previous chapters philosophies to the solutions design, can be a challenge but the following topics can help guide the engineer through the process of defining and using the discussed principles.

8.1. Graphic Form Layout

When starting the front-end design process, it is important to understand how the different areas of the screen are to be used.

As an example, the graphic form shown below is broken down into 4 areas. Each areas have a specific function or purpose.



Figure 8 – Graphic Form Layout

- **Main Navigation** – The navigation at the top of the SCADA interface allows the user to open different areas of the solution, as well as giving the user access to global/overview views and security options.
- **Graphic Form Navigation** – The navigation on the left will be specific to the currently opened Graphic Form. This navigation will allow the user to open alarm views, trends, etc related the currently open Graphic Form
- **TGO control** – any Graphic Form that is not opened as a modal form will be loaded into this area. Typically, this is where alarm view, trends, etc will be loaded by clicking on appropriate **Graphic Form Navigation** icon.
- **Main Display** – This area is where the SCADA Graphic Forms will load.
To help our customers Adroit Technologies have various Navigation Templates that can be downloaded off the website and used as a starting point for any project. From simple to complex multi-level solutions.

8.1.1. Graphic Form Sizing

Make sure to understand the UI environments that are to be used on the project i.e. Computer and mobile device screen resolutions. So if most of the devices are typically PC workstations the screen size of the Graphic Forms should be FHD i.e. "1920 x 1080"

However, if the devices are a type of mobile device the screen resolution can vary it is recommended to set constraints on the number of devices screen resolutions.

Some of the most typically device screen resolution are:

- 720×1600 (HD+): Popular for budget phones in emerging markets.
- 1080×2400 (FHD+): The workhorse resolution for mid-range and premium devices.
- 1440×3200 (QHD+): High-end flagships mobile devices

8.1.2. Navigation Hierarchy

A concept of hierarchy (Levels) should be followed in constructing navigation. The primary purpose of these levels is to provide different amounts of operating detail to aid the Operator in performing different tasks. A secondary purpose of these levels is to allow easier navigation. Four levels are optimum.

They are:

- **Level 1** – Overview of the Enterprise and/or Site.
- **Level 2** – Displays an Area within the selected Site.
- **Level 3** – Opens a modeless Graphic Form that displays details/state and allows supervisory control of the selected device or equipment.
- **Level 4** – Opens a modal Graphic Form that allows the user to change setpoints and set high level control.

8.2. Fonts

By setting standard font families, sizes, and weights for headings, body text, and labels, engineers ensure that content is visually organized and easy to scan. Consistent typography helps users quickly recognize different types of information and understand the hierarchy within the interface. It also supports accessibility by maintaining legibility across various devices and screen sizes. Defining default fonts reduces ambiguity in design decisions, improves user comprehension, and contributes to a clean, professional appearance throughout the application.

As an example:

Application	Font	Size
Default	Tahoma	10pt
Main Heading	Tahoma	16pt (Bold)
Sub Heading	Tahoma	12pt

8.3. Colours

Default colours are crucial to define in UI design because they establish a consistent visual language that guides user interaction. By setting clear default colours for elements like buttons, links, backgrounds and text, engineers ensure a cohesive and predictable experience across the interface.

This consistency helps users quickly understand functionality, such as which buttons are clickable or which fields require attention, without confusion. Additionally, default colours serve as a foundation for accessibility, ensuring sufficient contrast and readability for all users.

Ultimately, well-defined default colours streamline design decisions, support usability, and contribute to an intuitive user experience.

8.3.1. Static Colours

Static colours are used to display headings labels. The static colours are not driven by any events or alarms. For example:

Property Name	Back Colour	Fore Colour
Main Background	White	-
Headings	Grey	White
Static and Dynamic Labels	White	Black
Setpoints	White	Blue

8.3.2. Dynamic Colours

Dynamic colours are driven by either an event or an alarm state or a combination of both. The dynamic colours are applied as follows

- Default object fill is <no fill>, i.e. if no event/states/alarms
- Alarms must appear on top of events.
- For maintenance event, disable all alarms.

For example:

States:

Property Name	Colour	Affected Area
Default		Fill
On (Running)		Fill
Off (But Available)		Fill
Opening	Flashing	Out Line
Closing	Flashing	Out Line
Interlocked		Fill
Maintenance		Fill

Alarms:

Property Name	Colour	Affected Area
Lowest Priority (P0)		Out Line
Low Priority (P1)		Out Line
Normal Priority (P2)		Out Line
High Priority (P3)		Out Line
Highest Priority (P4)	Flashing	Out Line

Note: Further details please see the chapter regarding [Alarming Strategy](#).